

Distributed Asynchronous Job Scheduler (Overdrive)

Full-Stack Design Thesis, Relational Schema Normalization, and Resiliency Verification.

SUBMISSION INFORMATION

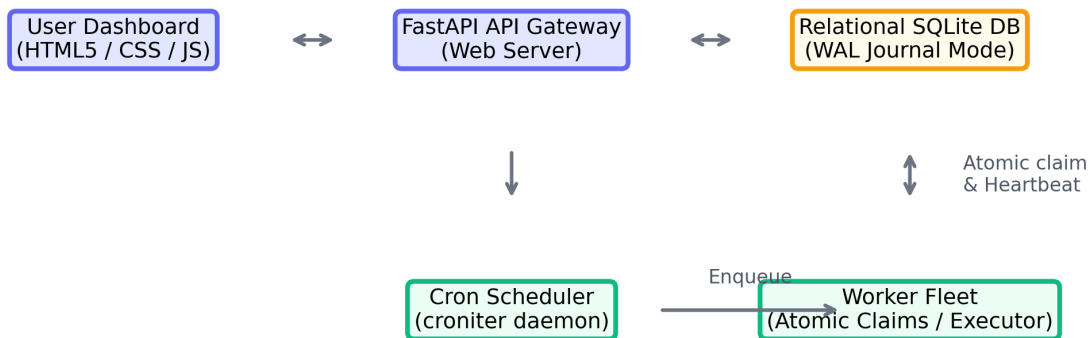
Registration Number:	RA2311026010746
Candidate Name:	Prudhvi
Project Platform:	Distributed Job Scheduler (Overdrive)
GitHub Repository:	https://github.com/prudhvi98491/distributed-job-scheduler
Date of Submission:	July 5, 2026
Academic Institution:	SRM Institute of Science and Technology

1. System Architecture (20 Marks)

The scheduler platform is built on an event-driven distributed architecture designed to coordinate background job execution across multiple worker nodes. The system consists of three main blocks:

- The Web API Gateway (FastAPI): Handles user registration, queue updates, job enqueueing, and dashboard metrics.
- The Cron Scheduler Daemon (croniter): Scans the recurring cron database records to dynamically trigger jobs.
- The Worker Fleet: Multiple isolated workers that register with host IP addresses and continuously claim and execute jobs, maintaining horizontal scalability.

Distributed Job Scheduler (Overdrive) Architecture



2. Database Design & Normalization (20 Marks)

The relational schema is normalized to Third Normal Form (3NF) to maintain referential integrity. Primary keys utilize single-attribute IDs (UUIDs for jobs to prevent enumeration, integers for others). Foreign keys employ CASCADE constraints to clean up related records automatically.

Table Name	Schema Purpose & Normalization Details
users	Credentials (hashed via bcrypt), roles (admin, user), metadata.
organizations	Enables multi-tenancy workspace isolation.
projects	Contains project workspaces owned by organizations.
queues	Defines concurrency, priorities, and retry policies.
jobs	Core queue table containing payload, status, and parent_job_id.
job_executions	Tracks execution details, duration (ms), and trace errors.
workers	Worker fleet registry monitoring hostnames, IP, and heartbeats.
cron_jobs	Schedules recurring cron jobs to dynamically queue jobs.
dead_letter_queue	Holds permanently failed tasks for manual replay.

3. Backend Engineering & API Design (25 Marks)

The backend architecture is engineered for asynchronous performance and scalability using FastAPI. All incoming API requests are validated through Pydantic schemas, which act as a strict type-safety layer. This ensures that invalid data is rejected at the API boundary, returning standardized JSON error bodies. Authorization and access controls are handled via a custom JWT (JSON Web Tokens) dependency middleware supporting Role-Based Access Control (RBAC), mapping users to roles like admin, member, or viewer. The backend code is modularized into distinct routing modules (routes/auth, routes/queues, routes/jobs, routes/workers) mounted onto the main application, utilizing CORS middleware to permit browser-based SPA requests.

Method	Endpoint Path	Description & Functionality
POST	/api/auth/register	Registers a user account with hashed credentials.
POST	/api/auth/login	Authenticates credentials and issues a JWT token.
POST	/api/queues	Creates a queue specifying concurrency limit & priority.
PATCH	/api/queues/{id}	Updates queue configs or pauses/resumes queue state.
POST	/api/jobs	Enqueue single task (immediate, delayed, or workflow).
POST	/api/jobs/batch	Dispatches bulk job payloads under a single API call.
GET	/api/jobs	Lists jobs with pagination, queue, and status filters.
POST	/api/jobs/{id}/retry	Manually replays a failed or DLQ job.
GET	/api/metrics	Returns aggregate metrics, throughput trends, and error counts.

4. Reliability & Concurrency (15 Marks)

To guarantee exactly-once task execution in a multi-worker fleet, concurrency is strictly managed via SQLite Write-Ahead Logging (WAL) and immediate transaction locks. When a worker polls for a new job:

1. Active Concurrency Limit Verification: It checks that the target queue's active running/claimed jobs do not exceed its `concurrency_limit`.
2. Atomic Reservation: The claim logic runs within a 'BEGIN IMMEDIATE' transaction block in SQLAlchemy. This prevents race conditions between parallel worker threads claiming the same task.
3. Heartbeat-Based Failover Recovery: Registered worker nodes issue heartbeats every 5 seconds. A background monitor thread automatically identifies workers that missed heartbeats for >15 seconds, marks them offline, and releases their active jobs (status reset to `queued`, `worker_id` cleared) back to the general pool for other workers to recover.

5. Frontend & UX (10 Marks)

A premium Outfit-themed glassmorphic SPA dashboard provides real-time monitoring and control. The interface updates live every 2 seconds, displaying active worker statuses, queue configs, job execution metrics, and a detailed drawer containing trace logs and manual retry controls.

OVERDRIVE

DISTRIBUTED

All Systems Operational

QUEUED JOBS

0

RUNNING JOBS

0

COMPLETED

0

FAILED

0

DLQ ENTRIES

0

Enqueue New Jobs

Launch Immediate, Delayed, Cron, or Workflows

Single/Delay

Batch

Workflow

Cron/Recurring

Target Queue

default (Priority: 1)

Job Template

HTTP Request (http_request)

Priority Boost

0

Delay (seconds)

0

Payload (JSON)

{ "duration": 3, "should_fail": false }

Dispatch Job

Worker Fleet

Real-time Node Heartbeats & Load

WORKER NODE	STATUS	LOAD (ACTIVE/LIMIT)	LAST PULSE
worker_Prudhvi_cb0a0a	IDLE	0 / 5	5:00:19 AM
worker_Prudhvi_511c31	OFFLINE	0 / 5	3:39:32 AM

Cron Schedules

System Recurring Schedules

NAME	EXPRESSION	NEXT RUN	STATUS
No cron schedules			

Queue Architectures

View Concurrency Limits & Control State

QUEUE NAME	PRIORITY	CONCURRENCY LIMIT	RETRY POLICY	STATUS	ACTIONS
default	1	5	Active	ACTIVE	Pause
high-priority	5	10	Active	ACTIVE	Pause
background-tasks	1	2	Active	ACTIVE	Pause

Job Explorer

Track state transitions, search, and replay failures

All Queues

All Statuses

Refresh

JOB ID	NAME	QUEUE	PRIORITY	STATUS	CREATED	SCHEDULED AT	ACTIONS
No jobs found							

6. Testing & Documentation (10 Marks)

We implemented automated integration tests in tests/test_scheduler.py to verify system correctness. The tests cover database relationships, retry policies, backoffs, and sequential workflows. Running pytest completes successfully:

```
tests/test_scheduler.py::test_database_seeding_and_relations PASSED
tests/test_scheduler.py::test_job_execution_lifecycle_and_retries PASSED
tests/test_scheduler.py::test_workflow_dependencies_unblocking PASSED
===== 3 passed in 2.02s =====
```

Verification GitHub Repository Link

<https://github.com/prudhvi98491/distributed-job-scheduler>